Kafka Post-Deployment Validation SOP

Objective

Validate Kafka producers, consumers, topics, and consumer groups after application or infrastructure deployment.

Scope

Applicable to Kafka-based applications, streaming platforms, backend services, data engineering pipelines, DevOps teams, and platform teams supporting production or synthetic incident workflows.

Pre-requisites

1. Access to Kafka cluster, namespace or broker host.
2. Access to monitoring dashboards such as Grafana, Prometheus, or cloud monitoring.
3. Kafka CLI tools: kafka-topics, kafka-consumer-groups, kafka-console-producer, and kafka-console-consumer.
4. Permission to view application logs, broker logs, and deployment status.
5. Escalation contact for DevOps, platform, and application owner teams.

Incident Metadata

Synthetic operational SOP derived from Kafka ticket and log patterns in the provided dataset. Use this document for alert triage, validation, and ticket enrichment scenarios.

Procedure

1. Confirm deployment version, impacted services, topics, and consumer groups.
2. Check producer send rate and producer error rate after deployment.
3. Check consumer lag, rebalance count, and timeout errors.
4. Validate topic metadata, partition leaders, and replication health.
5. Review logs for Kafka error codes generated during the deployment window.
6. Send or trace a test event where safe and approved.
7. Rollback or scale only if metrics breach defined thresholds.
8. Update deployment notes with Kafka validation result and any follow-up action.

Common Commands

```
kafka-topics.sh --bootstrap-server <broker>:9092 --describe
kafka-consumer-groups.sh --bootstrap-server <broker>:9092 --describe --group <group>
kafka-broker-api-versions.sh --bootstrap-server <broker>:9092
kubectl get pods -n <namespace> | grep kafka
kubectl logs <kafka-pod> -n <namespace> --tail=100
```

Troubleshooting Matrix

Issue	Possible Cause	Resolution
Producer errors after deploy	Bad config or ACL	Validate bootstrap, topic, and ACL
Lag spike after deploy	Consumer slowdown	Scale or rollback after analysis
Rebalance spike	Consumer restart pattern	Tune deployment strategy
Replication warning	Broker or topic issue	Validate cluster health

Best Practices

1. Include Kafka checks in release readiness.
2. Do not rely only on application health checks.

3. Keep canary event validation for critical streams.
4. Review Kafka dashboards for at least 15 minutes after deploy.

References

Apache Kafka Docs | Internal Logs CSV | Tickets CSV | Monitoring Dashboards | Kafka CLI Runbook